Spring Boot 小游戏 Docker 容器化实验报告

记忆花园 memory-game6085

课程：2026 云计算技术与架构

姓名：周宇斌

班级：软件 2401

学号：20246085

实验日期：2026 年 10 月 21 日



图 1 记忆花园容器运行页面

本实验将 Vue3、Spring Boot 与 SQLite 记忆翻牌小游戏打包为 Docker 镜像，完成本机仓库发布、删除本地镜像后拉取和数据卷保留验证。

目录

一 实验目的与任务要求 3
二 实验环境与项目结构 4
2.1 实验配置与项目目录 4
2.2 环境启动与验证 5
三 系统设计与持久化 6
3.1 应用结构 6
3.2 SQLite 数据设计 6
3.3 容器数据卷 6
四 本地构建与功能验证 7
4.1 本地构建与自动验证 7
4.2 容器中的游戏操作与成绩保存 7
五 Docker 镜像构建与运行 8
5.1 多阶段构建 8
5.2 JAR 打包与镜像导出 9
5.3 启动容器与接口检查 10
六 镜像发布与拉取 11
6.1 启动本机镜像仓库 11
6.2 标记与推送镜像 11
6.3 删除本地镜像后重新拉取 12
七 容器重建与持久化验证 14
7.1 记录重建前的数据 14
7.2 使用拉取镜像创建新容器 15
7.3 对比重建后的成绩 16
八 实验结果与问题分析 17
8.1 启动与端口问题 17
8.2 实验总结 17
九 参考资料与证据来源 18
附录 A Docker 安装与配置补充实验 19
A1 安装前检查 20
A2 配置 Docker 官方软件源 21
A3 安装 Docker Engine 和插件 22
A4 首次配置与引擎启动 24
A5 导入基础镜像并验证安装 25
A6 创建网络、数据卷和镜像仓库 26
A7 构建应用镜像并核对 JAR 27
A8 配置应用容器并保存成绩 29
A9 标记并发布镜像 31
A10 删除本地标签后重新拉取 32
A11 重建容器并核对持久化数据 33

一 实验目的与任务要求

本实验的目的是掌握 Spring Boot 应用容器化的基本过程，理解 Docker 镜像、容器、镜像仓库和持久化数据卷的作用，并通过实际命令和访问结果验证部署流程。

在机器上部署 Docker 环境，将一个已有 Spring Boot 项目打包到 Docker 上发布，项目名称末尾加学号后四位；发布后演示拉取，所有过程截图放在一个 Word 文件中提交。

任务要求	本实验对应方式
项目名称包含学号后四位	memory-game6085，学号后四位为 6085
Spring Boot 项目打包发布	Windows 多阶段构建见第五章；补充环境的配套 JAR 镜像构建见附录 A7。
发布后演示拉取	Windows 仓库发布与拉取见第六章；补充环境的发布与拉取见附录 A9 至 A10。
全过程截图汇总	正文图 1 至图 11，以及附录图 A1 至图 A11（含分图），均嵌入本 Word。
Docker 环境部署与验证	Windows 版本验证见 2.2；补充环境的安装、首次配置、验证及资源创建见附录 A1 至 A6。

二 实验环境与项目结构

第二至九章记录 2026 年 10 月 21 日在 Windows 上完成的实验。附录 A 记录 10 月 22 日在本机隔离 Ubuntu 环境中安装 Docker Engine 并部署应用的过程。两次实验使用不同的引擎和资源。

2.1 实验配置与项目目录

项目	本次实际配置
操作系统	Windows 11 家庭版 中文版 64 位，10.0.26300
本地开发工具	Java 21.0.11；Node.js 24.14.1；npm 11.11.0；Maven 3.9.16
应用技术栈	Vue 3.5.44、Vite 7.3.7、Spring Boot 4.1.1、SQLite JDBC 3.53.4.0
Docker 环境	Docker Desktop 4.95.0；Client / Server 29.8.2；Compose 5.6.0
构建与运行镜像	Node 24；Maven 3.9.9 + Java 21；Temurin 21 JRE
源码目录	Code/memory-game6085/
应用端口	宿主机 127.0.0.1:18085 → 容器 8080
镜像仓库	本机 Registry，127.0.0.1:5000
持久化目录	memory-game6085-data → /app/data，数据库 garden.db

frontend 保存 Vue 页面和图案，backend 保存 Spring Boot 接口及数据库逻辑，scripts 保存本地构建与实验辅助脚本。Dockerfile 与 compose.yaml 位于项目根目录。本实验采用 18085 作为宿主机端口。

2.2 环境启动与验证

本实验沿用 Windows 上已安装的 Docker Desktop，使用 WSL2 的 Linux 容器引擎。原有 WSL 数据目录继续使用。表中的环境版本取自本次实验机器，图 2 展示恢复启动后的实际验证结果。

启动时遇到遗留 socket 故障，Docker Desktop 无法正常提供服务。为恢复启动，在 Docker Desktop 停止时执行下面的辅助脚本，仅为该子进程设置独立的 LOCALAPPDATA 运行时缓存目录，保留系统级环境变量。

.\scripts\start-docker-runtime.ps1

引擎启动后，在项目根目录执行版本与命名检查。docker version 同时返回 Client 和 Server 29.8.2，Engine 为 linux/amd64；Compose 返回 5.6.0，项目目录为 memory-game6085。随后执行第五章的构建与运行操作。

原 Windows 环境沿用既有安装，历史安装过程截图未提供。附录 A 补充 2026 年 10 月 22 日在本机隔离 Ubuntu 环境中实际安装 Docker Engine、首次配置以及网络、数据卷和容器创建的操作记录。

docker version

docker compose version

Get-Item . | Select-Object Name, FullName

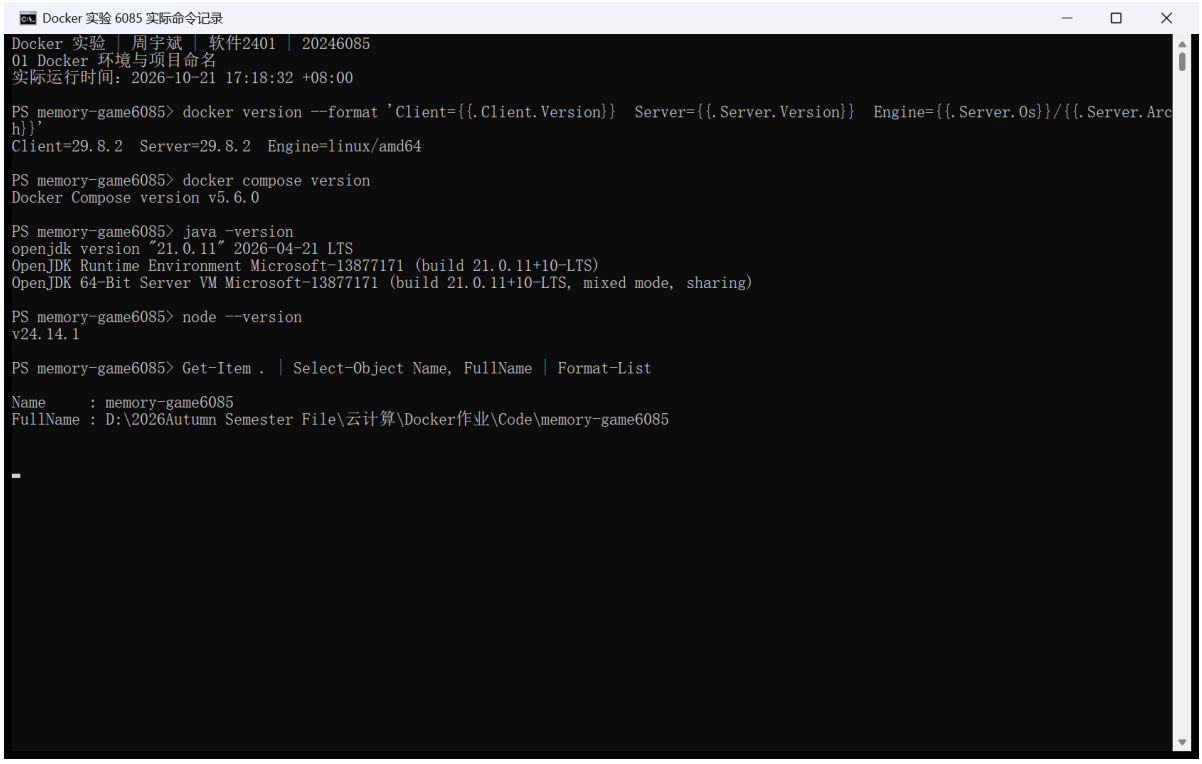


图 2 Docker Client 与 Server 版本及项目名称检查

三 系统设计与持久化

3.1 应用结构

浏览器通过 Vue3 页面操作游戏，Spring Boot 提供牌局、翻牌、暂停、成绩提交和排行榜接口。前端构建结果复制到后端静态资源目录，再打入可执行 JAR，因此一个应用容器即可同时提供页面和接口。

游戏提供轻松 6 对、标准 8 对和挑战 12 对三种难度。后端随机生成卡牌顺序并判断配对，负责步数和计时。未翻开的答案不会直接传给浏览器；暂停时保留本局状态并停止计时，恢复后继续游戏。

3.2 SQLite 数据设计

字段	用途
game_id	牌局唯一标识，防止同一局重复写入
nickname / difficulty	玩家昵称与难度，按难度分别查询排行榜
moves / duration_ms	服务端生成的步数与用时，作为排序依据
created_at	成绩记录创建时间

运行中的牌局保存在服务端内存中；服务重启后需要重新开局。已提交成绩保存在 SQLite 数据库，排行榜按步数升序排列，步数相同再比较用时。持久化对象是已保存成绩，不包括未完成牌局。

3.3 容器数据卷

```
volumes:
  - garden-data:/app/data
# 命名数据卷的实际名称
name: memory-game6085-data
```

镜像保存应用程序，数据库由运行时生成并写入 /app/data。该目录挂载到命名数据卷后，删除应用容器不会自动删除成绩文件。重建容器时继续挂载同一数据卷，检查已保存的成绩是否仍可读取。

四 本地构建与功能验证

4.1 本地构建与自动验证

.\scripts\build.ps1
java -jar backend/target/memory-game6085.jar

本地构建已生成可执行 JAR。后端测试覆盖隐藏牌面、无效操作、暂停状态、服务端成绩统计和 SQLite 重新打开后的数据保留。浏览器测试覆盖完整配对、昵称校验、成绩提交、榜单刷新、窄屏布局以及接口失败后的重试。

检查项目	结果
npm run build	成功生成前端静态资源
后端测试	3 项通过，0 失败，0 错误
浏览器端到端测试	3 项全部通过
界面检查	在 1440、768、375、812（横屏）及 320 px 宽度下均无横向溢出。
数据保存	完成牌局后可填写昵称并保存成绩，成绩会显示在对应难度的排行榜中。

4.2 容器中的游戏操作与成绩保存

通过浏览器在应用容器中完成轻松难度的一局游戏，并以昵称“6085 实验”保存成绩。服务端返回的记录编号为 1，成绩为 12 步、6.95 秒，用于演示成绩保存流程。

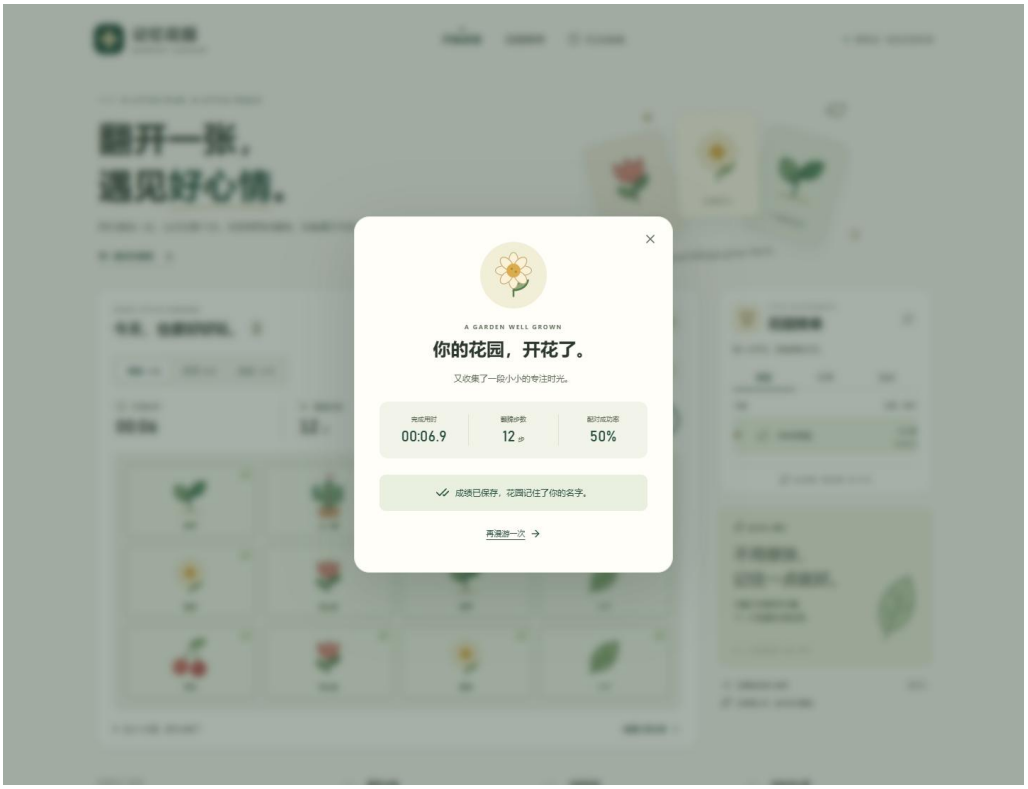


图 3 容器中完成配对并保存演示成绩

五 Docker 镜像构建与运行

5.1 多阶段构建

第一阶段使用 Node 构建 Vue，第二阶段使用 Maven 编译和打包 Spring Boot，第三阶段只复制 JAR 到 Java 运行环境。以下为实际 Dockerfile 的关键流程；完整文件位于项目根目录。

```
FROM node:24-bookworm-slim AS frontend
WORKDIR /build/frontend
COPY frontend/package*.json ./
RUN npm ci --no-audit --no-fund
COPY frontend/ ./
RUN npm run build
FROM maven:3.9.9-eclipse-temurin-21 AS backend
WORKDIR /build/backend
COPY backend/pom.xml ./
RUN mvn -B -ntp dependency:go-offline
COPY backend/src ./src
COPY --from=frontend /build/frontend/dist /build/frontend/dist
RUN mvn -B -ntp package
FROM eclipse-temurin:21-jre-jammy
# 创建普通用户与数据目录后复制 JAR
COPY --from=backend --chown=garden:garden /build/backend/target/memory-game6085.jar /app/app.jar
USER garden
ENV DATA_DIR=/app/data PORT=8080
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=70.0", "-jar", "/app/app.jar"]
```

5.2 JAR 打包与镜像导出

docker compose --progress plain build

构建日志显示 Maven BUILD SUCCESS，最终镜像命名为 memory-game6085:1.0.0。Docker CLI 显示磁盘占用约 469 MB。首次基础镜像下载曾发生短读错误，完成基础镜像拉取后重新构建成功。

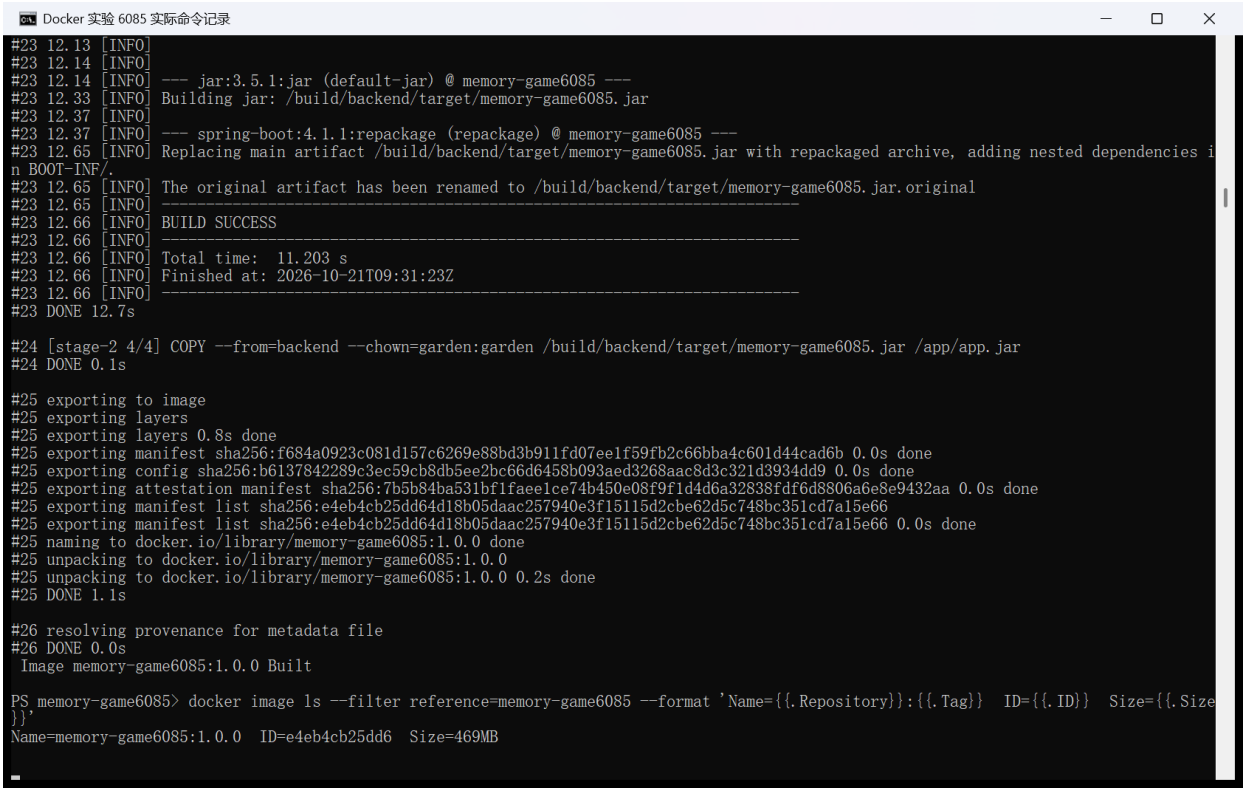


图 4 Spring Boot JAR 打包成功并导出 Docker 镜像

5.3 启动容器与接口检查

docker compose up -d --no-build
docker compose ps
curl.exe -s http://localhost:18085/api/health
docker exec memory-game6085-garden-1 id

应用容器运行后，18085 端口可访问游戏和 API。健康检查返回 status 为 UP，database 为 SQLite。容器内运行身份为 garden，UID 和 GID 均为 999，应用不使用 root 用户运行。

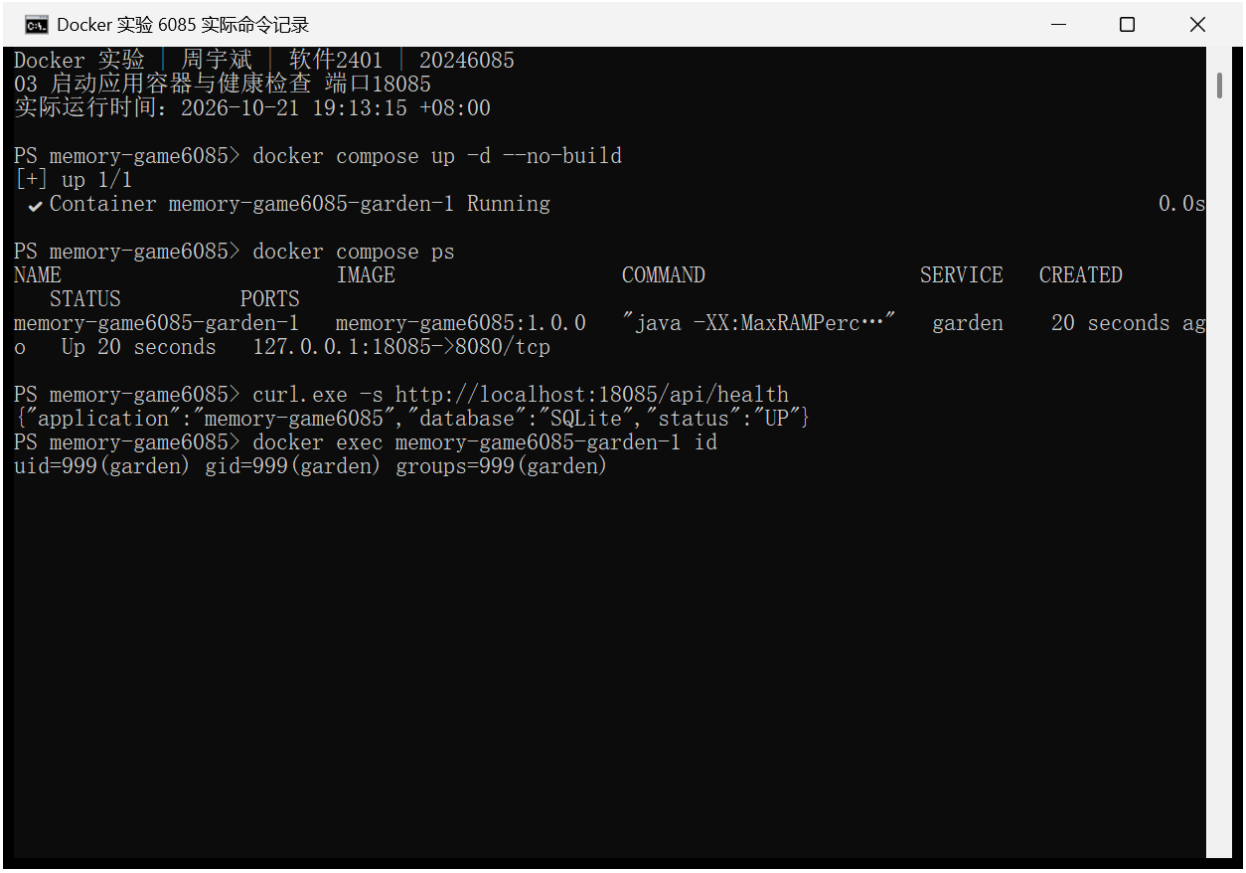


图 5 应用容器运行状态、端口映射、健康检查与运行用户

浏览器访问 http://localhost:18085 可进入游戏。该地址映射至应用容器的 8080，前端和后端同源访问，SQLite 无需额外数据库容器。

六 镜像发布与拉取

6.1 启动本机镜像仓库

本实验使用本机私有 Registry，仓库端口绑定 127.0.0.1:5000，镜像数据保存在 memory-game6085-registry-data 数据卷。仓库用于本机课程实验。

```
docker run -d --name memory-game6085-registry -p 127.0.0.1:5000:5000 -e OTEL_TRACES_EXPORTER=none --mount
type=volume,source=memory-game6085-registry-data,target=/var/lib/registry registry:3
```

6.2 标记与推送镜像

```
docker tag memory-game6085:1.0.0 localhost:5000/memory-game6085:1.0.0
docker push localhost:5000/memory-game6085:1.0.0
curl.exe -s http://localhost:5000/v2/memory-game6085/tags/list
```

推送日志显示镜像层 Pushed，仓库标签查询返回 1.0.0。发布路径为 localhost:5000/memory-game6085:1.0.0。

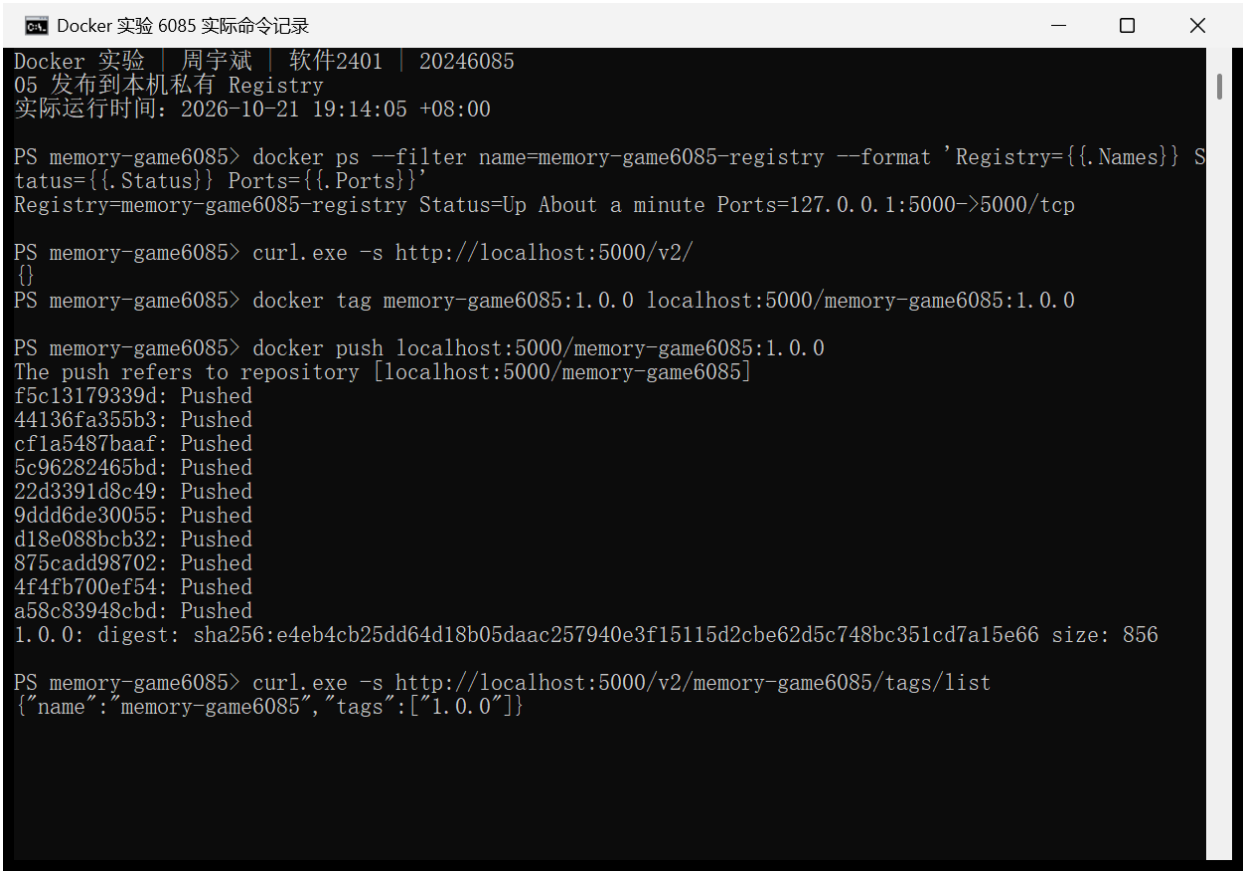


图 6 镜像推送成功与仓库标签查询

6.3 删除本地镜像后重新拉取

为验证镜像来自仓库，先移除本实验应用容器，保留数据卷，并删除本地的两个实验镜像标签。镜像列表确认本地不存在 memory-game6085 后，再执行 pull。

docker compose down

docker image rm localhost:5000/memory-game6085:1.0.0 memory-game6085:1.0.0

docker image ls

docker pull localhost:5000/memory-game6085:1.0.0

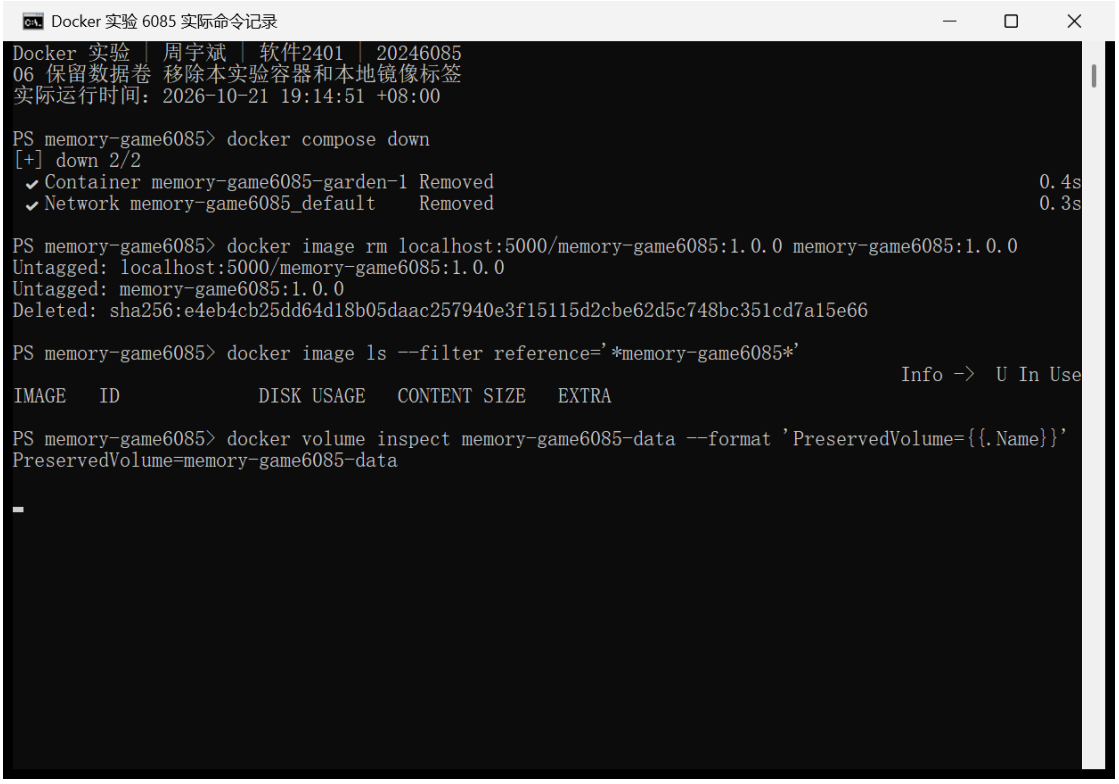


图 7 移除实验容器和本地镜像，数据卷仍然保留

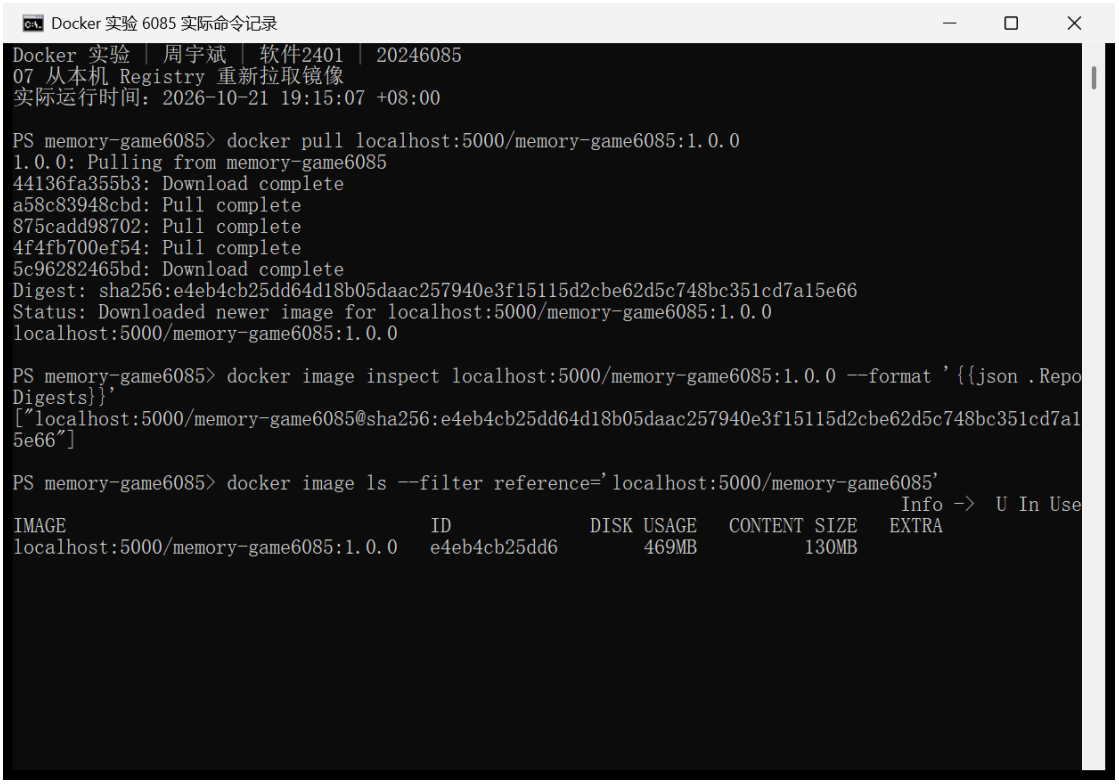


图 8 从 Registry 重新拉取成功

拉取结果为 Downloaded newer image。推送与拉取的摘要一致，说明使用的是已发布的同一版本镜像。摘要记录如下：

sha256:e4eb4cb25dd64d18b05daac257940e3f15115d2cbe62d5c748bc351cd7a15e66

七 容器重建与持久化验证

7.1 记录重建前的数据

原容器中已保存轻松难度演示成绩。实验记录了原容器标识和完整排行榜响应；移除容器时未使用删除数据卷的参数。

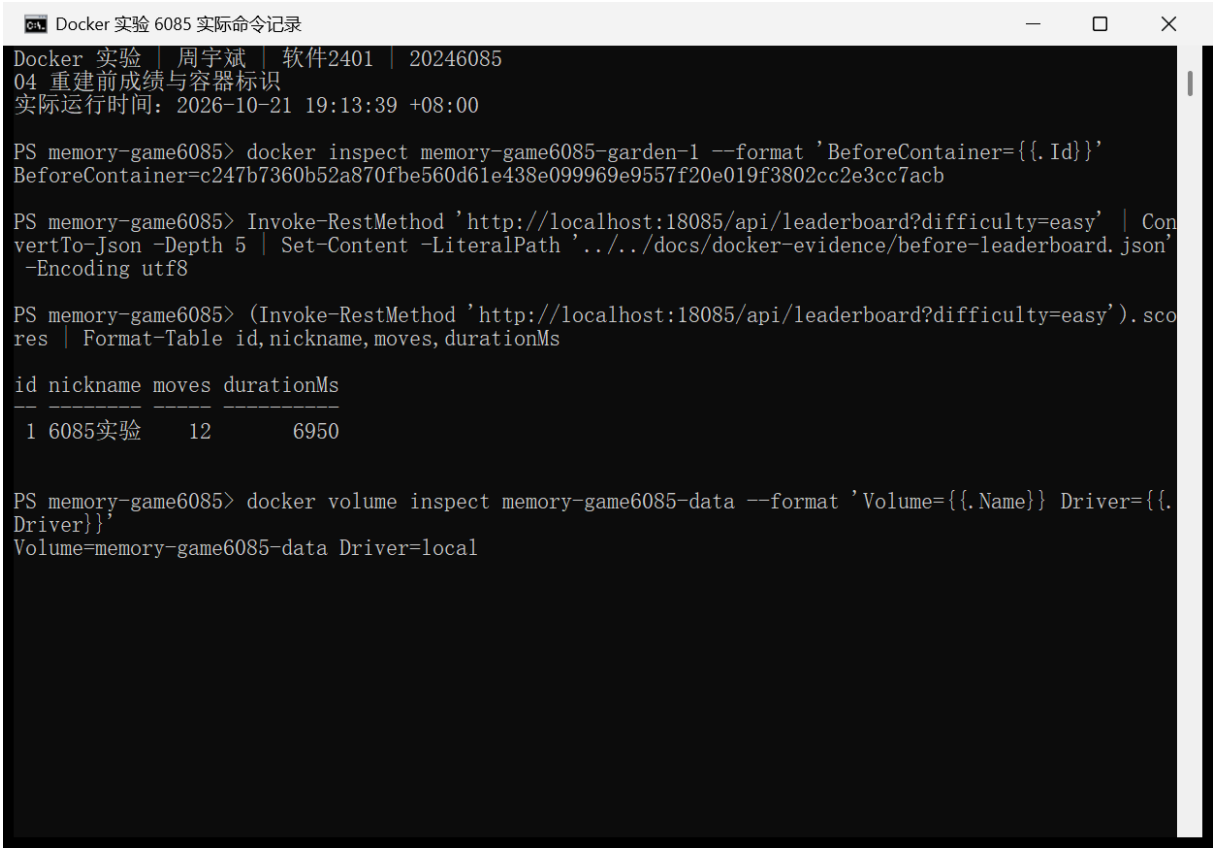


图 9 重建前的容器标识、成绩与数据卷

对比项	重建前的实际值
容器标识前 12 位	c247b7360b52
成绩编号 / 昵称	1 / 6085 实验
步数 / 用时	12 / 6950 ms
数据卷	memory-game6085-data

7.2 使用拉取镜像创建新容器

docker run -d --name memory-game6085-pulled -p 127.0.0.1:18085:8080 -e DATA_DIR=/app/data --mount type=volume,source=memory-game6085-data,target=/app/data localhost:5000/memory-game6085:1.0.0

新容器标识前 12 位为 dd8b4bb17b50，与原容器不同。新容器直接使用拉取的仓库镜像，并继续挂载 memory-game6085-data 到 /app/data。

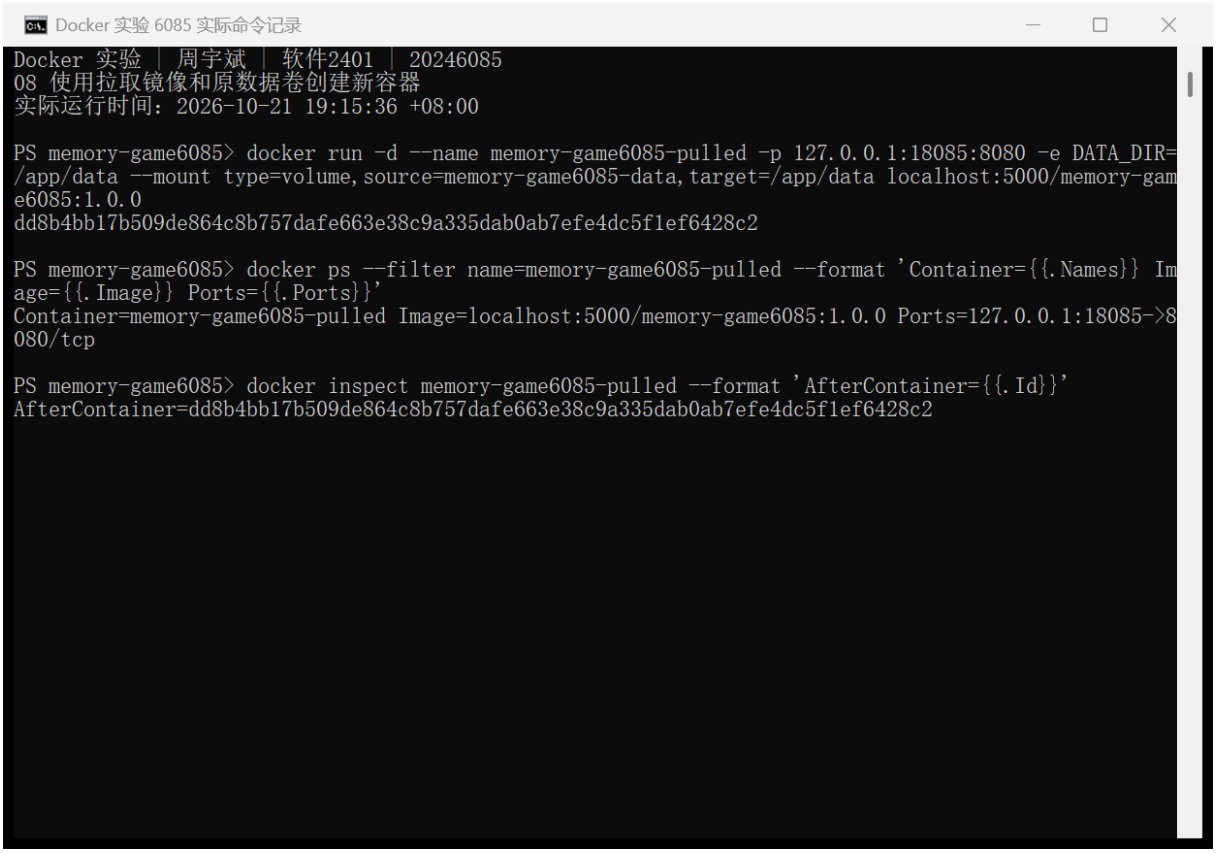


图 10 使用拉取镜像创建新容器并确认容器标识

7.3 对比重建后的成绩

新容器启动后，成绩编号、昵称、难度、步数、用时和创建时间均与重建前一致。程序比较了重建前后的完整排行榜 JSON，结果为 LeaderboardPreserved=True。记录仍为 6085 实验、12 步、6950 ms。

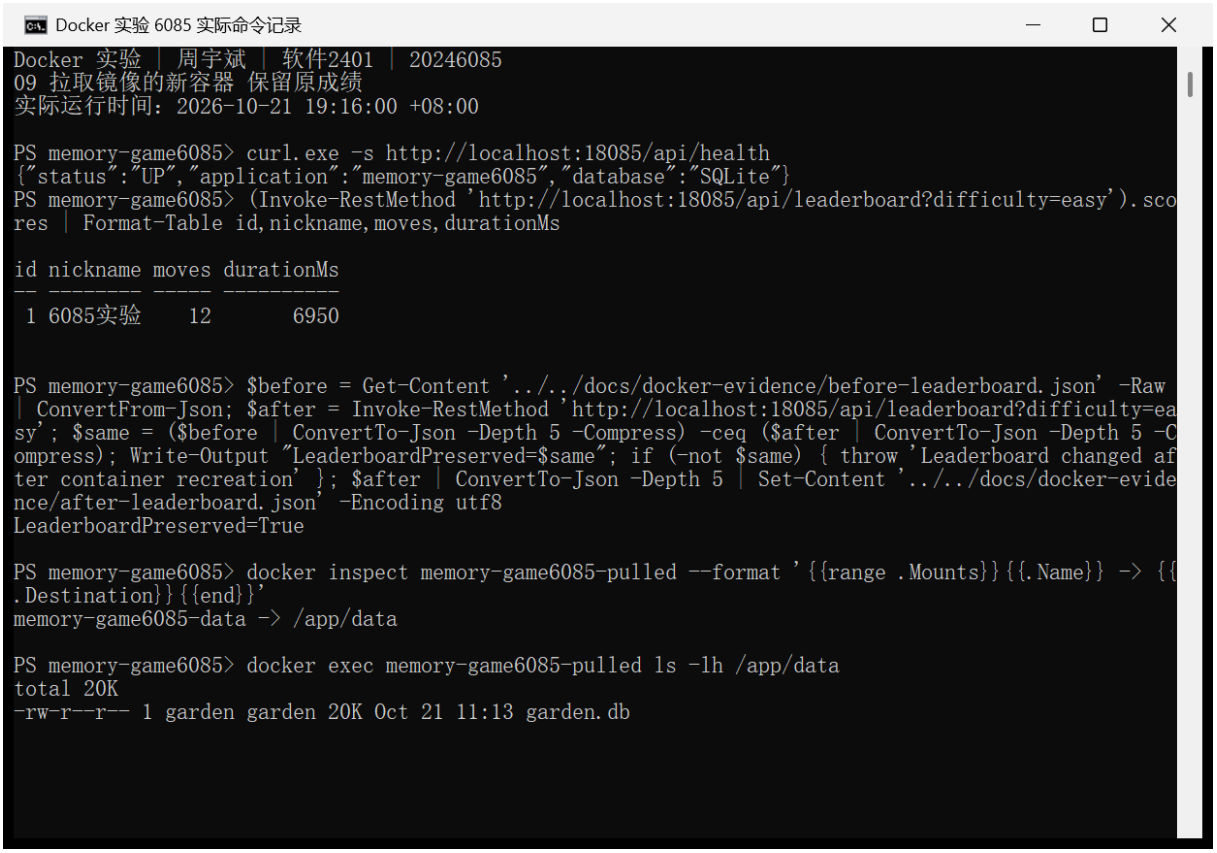


图 11　重建后榜单一致，数据卷仍挂载到同一目录

八 实验结果与问题分析

实验环节	实际结果
Docker 环境	Client / Server 正常，Linux 引擎可用
应用与镜像构建	前端构建、后端测试和 Spring Boot JAR 打包成功
容器运行	页面和 API 可访问，健康检查 UP
镜像发布	本机 Registry push 成功，可查询标签 1.0.0
镜像拉取	删除本地镜像后 pull 成功，摘要与推送一致
容器重建	新容器使用拉取镜像，容器标识与原容器不同
SQLite 数据卷	重建前后的完整榜单一致，已保存成绩未丢失
实验截图	真实终端窗口与容器浏览器页面均已采集

8.1 启动与端口问题

Docker Desktop 最初因遗留 socket 无法访问而退出。原运行时目录改名被 Windows 拒绝，最终通过仅为 Docker Desktop 子进程设置独立运行时缓存目录恢复启动，系统级环境变量未修改。恢复后，Client 与 Server 均能响应，容器也可正常运行。

首次构建遇到基础镜像下载中断；重新拉取所需基础镜像后完成构建。现有课堂项目占用 8080，因此将本实验宿主机端口调整为 18085，同时保留容器内部端口 8080。

8.2 实验总结

本实验将 Spring Boot 项目打包为镜像，推送到本机 Registry，并在删除本地应用镜像标签后重新拉取。使用拉取的镜像创建新容器后，页面和接口可正常访问。

重建前后，容器标识发生变化，已保存的成绩保持不变，说明容器重建后仍能读取原数据卷中的 SQLite 数据。后续复现实验时，应保持仓库地址、镜像版本、应用端口和数据卷名称一致。

九 参考资料与证据来源

实验项目为记忆花园 memory-game6085，采用 Vue3、Spring Boot 和 SQLite，镜像仓库为本机 Registry。

源码包解压后的项目目录为 memory-game6085/。正文图 1 至图 11 来自原 Windows 实验，其结果记录在 docs/docker-evidence/results.json；附录 A 来自本轮补充实验，命令、完整日志和原图记录在 docs/v7-work/。报告中的过程截图均为内嵌图片。

正文技术资料查阅日期为 2026 年 10 月 21 日；Docker Engine 安装文档核对日期为 10 月 22 日。

Vue 官方文档：https://vuejs.org/guide/essentials/reactivity-fundamentals.html

Vite 官方文档：https://vite.dev/guide/

Spring Boot 官方文档：https://docs.spring.io/spring-boot/maven-plugin/packaging.html

SQLite JDBC 官方仓库：https://github.com/xerial/sqlite-jdbc

Docker 数据卷官方文档：https://docs.docker.com/engine/storage/volumes/

CNCF Distribution 官方部署文档：https://distribution.github.io/distribution/about/deploying/

Docker Engine 官方 Ubuntu 安装文档：https://docs.docker.com/engine/install/ubuntu/

附录 A Docker 安装与配置补充实验

本附录记录从未安装 Docker 的 Ubuntu 环境开始，依次完成软件源配置、Docker Engine 安装、首次配置、资源创建和应用发布拉取。原 Windows 实验与本轮补充实验使用不同引擎和资源。

外层环境是本机 Docker Desktop，提供隔离 Ubuntu 容器；Docker Engine 29.9.2 安装在该容器内部，未挂载宿主机 Docker socket。后续应用镜像、网络、数据卷和 Registry 均由新安装的内层引擎管理。该环境用于课程补充实验。

图中的安装和命令操作记录为实际执行日志在浏览器中的截屏，按原日志顺序保留命令、输出与退出码；游戏页面图直接采集自本轮运行的应用容器。日志中的时间使用 UTC，与北京时间相差 8 小时。

项目	本轮补充实验实际配置
安装系统	Ubuntu 24.04.5 LTS，amd64；全新隔离环境内安装
Docker 组件	Engine / CLI 29.9.2；Compose 5.6.1 Buildx 0.38.1；containerd.io 2.4.2
引擎配置	数据目录 /var/lib/docker-lab6085；vfs；json-file 日志轮转
应用与仓库	镜像 memory-game6085:1.0.2-v7；内层 Registry localhost:5000
网络和数据卷	memory-game6085-v7-net memory-game6085-v7-data memory-game6085-v7-registry
访问路径	本机 127.0.0.1:18089 → 隔离环境 18085 → 应用容器 8080
应用 JAR	使用源码包中的配套 JAR；容器内文件与配套 JAR 的 SHA-256 相同，逐字节比较也一致。
演示成绩	6085 补充实验；编号 1；10 步；4737 ms
重建验证	186285d65c93 → 00a18a115fb4；完整排行榜一致

A1 安装前检查

在新建的 Ubuntu 环境中检查系统版本、CPU 架构、Docker 命令和 socket。检查时 docker、dockerd 尚未安装，/var/run/docker.sock 不存在。

Docker 6085 补充实验 · 实际命令日志

01-before-install.txt · 原日志第 1—28 行 · UTC 时间按日志保留

```
Actual command: docker exec memory-game6085-v7-install-20261022 bash /lab/01-before-install.sh
Started UTC: 2026-10-22T04:53:35.963679+00:00

+ date -Iseconds
2026-10-22T04:53:36+00:00
+ cat /etc/os-release
+ head -6
PRETTY_NAME="Ubuntu 24.04.5 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.5 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
+ uname -m
x86_64
+ command -v docker
+ echo 'Docker CLI: not installed in this fresh Ubuntu environment'
+ command -v dockerd
Docker CLI: not installed in this fresh Ubuntu environment
+ echo 'Docker Engine: not installed'
+ test -S /var/run/docker.sock
+ echo 'No Docker socket; host Docker socket is NOT mounted'
+ dpkg-query -W docker-ce docker-ce-cli containerd.io
Docker Engine: not installed
No Docker socket; host Docker socket is NOT mounted
+ true

Actual exit code: 0
```

图 A1 安装前检查（操作日志）

A2 配置 Docker 官方软件源

先安装 ca-certificates 和 curl，再保存 Docker 官方 GPG 公钥，添加 noble stable 软件源并更新软件包索引。apt-cache 显示此时 Docker CE 尚未安装，候选版本为 29.9.2。

Docker 6085 补充实验 · 实际命令日志

02-official-repository.txt · 原日志第 1—31 行 · UTC 时间按日志保留

```
Actual command: docker exec memory-game6085-v7-install-20261022 bash /lab/02-official-repository.sh
Started UTC: 2026-10-22T04:53:36.533086+00:00

+ export DEBIAN_FRONTEND=noninteractive
+ DEBIAN_FRONTEND=noninteractive
+ date -Iseconds
2026-10-22T04:53:36+00:00
+ apt-get update
+ apt-get install -y ca-certificates curl
+ install -m 0755 -d /etc/apt/keyrings
+ curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
+ chmod a+r /etc/apt/keyrings/docker.asc
+ cat
+ cat /etc/apt/sources.list.d/docker.sources
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: noble
Components: stable
Architectures: amd64
Signed-By: /etc/apt/keyrings/docker.asc
+ apt-get update
+ tail -8 /lab/02-docker-repository-full.log
Get:1 https://download.docker.com/linux/ubuntu noble InRelease [48.5 kB]
Get:2 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [86.7 kB]
Hit:3 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:5 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:6 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Fetched 135 kB in 2s (63.8 kB/s)
Reading package lists...
+ apt-cache policy docker-ce
```

图 A2a 配置 Docker 官方软件源（操作日志，第 1 部分）

Docker 6085 补充实验 · 实际命令日志

02-official-repository.txt · 原日志第 32—44 行 · UTC 时间按日志保留

```
+ head -10
docker-ce:
  Installed: (none)
  Candidate: 5:29.9.2-1~ubuntu.24.04~noble
  Version table:
     5:29.9.2-1~ubuntu.24.04~noble 500
        500 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages
     5:29.9.1-1~ubuntu.24.04~noble 500
        500 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages
     5:29.9.0-1~ubuntu.24.04~noble 500
        500 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages

Actual exit code: 0
```

图 A2b 配置 Docker 官方软件源（操作日志，第 2 部分）

A3 安装 Docker Engine 和插件

通过 apt-get 安装 docker-ce、docker-ce-cli、containerd.io、Buildx 和 Compose 插件。软件包配置完成后，分别查看包版本及 docker、dockerd、Compose 的版本，实际安装 Docker Engine 29.9.2 和 Compose 5.6.1。

Docker 6085 补充实验 · 实际命令日志

03-install-engine.txt · 原日志第 1—31 行 · UTC 时间按日志保留

```
Actual command: docker exec memory-game6085-v7-install-20261022 bash /lab/03-install-engine.sh
Started UTC: 2026-10-22T04:55:33.471383+00:00

+ export DEBIAN_FRONTEND=noninteractive
+ DEBIAN_FRONTEND=noninteractive
+ date -Iseconds
2026-10-22T04:55:33+00:00
+ apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-
plugin
+ tail -22 /lab/03-apt-install-full.log
update-alternatives: using /usr/sbin/arptables-nft to provide /usr/sbin/arptables (arptables) in
auto mode
update-alternatives: using /usr/sbin/ebtables-nft to provide /usr/sbin/ebtables (ebtables) in auto
mode
Setting up openssh-client (1:9.6p1-3ubuntu13.20) ...
Setting up python3.12 (3.12.3-1ubuntu0.17) ...
Setting up libxext6:amd64 (2:1.3.4-1build2) ...
Setting up liberror-perl (0.17029-2) ...
Setting up dbus-user-session (1.14.10-4ubuntu4.1) ...
Setting up docker-ce (5:29.9.2-1~ubuntu.24.04~noble) ...
invoke-rc.d: could not determine current runlevel
invoke-rc.d: policy-rc.d denied execution of start.
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service →
/usr/lib/systemd/system/docker.service.
Created symlink /etc/systemd/system/sockets.target.wants/docker.socket →
/usr/lib/systemd/system/docker.socket.
Setting up git (1:2.43.0-1ubuntu7.4) ...
Setting up xauth (1:1.1.2-1build1) ...
Setting up docker-ce-rootless-extras (5:29.9.2-1~ubuntu.24.04~noble) ...
Setting up libpython3-stdlib:amd64 (3.12.3-0ubuntu2.1) ...
Setting up python3 (3.12.3-0ubuntu2.1) ...
Setting up python3-gi (3.48.2-1) ...
Setting up python3-dbus (1.3.2-5build3) ...
Setting up networkd-dispatcher (2.2.4-1) ...
Created symlink /etc/systemd/system/multi-user.target.wants/networkd-dispatcher.service →
/usr/lib/systemd/system/networkd-dispatcher.service.
Processing triggers for libc-bin (2.39-0ubuntu8.9) ...
```

图 A3a 安装 Docker Engine 和插件（操作日志，第 1 部分）

Docker 6085 补充实验 · 实际命令日志

03-install-engine.txt · 原日志第 32—45 行 · UTC 时间按日志保留

```
+ dpkg-query -W docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
containerd.io   2.4.2-3~ubuntu.24.04~noble
docker-buildx-plugin    0.38.1-1~ubuntu.24.04~noble
docker-ce       5:29.9.2-1~ubuntu.24.04~noble
docker-ce-cli   5:29.9.2-1~ubuntu.24.04~noble
docker-compose-plugin   5.6.1-1~ubuntu.24.04~noble
+ docker --version
Docker version 29.9.2, build a82e528
+ dockerd --version
Docker version 29.9.2, build 0367cbe
+ docker compose version
Docker Compose version v5.6.1

Actual exit code: 0
```

图 A3b 安装 Docker Engine 和插件（操作日志，第 2 部分）

A4 首次配置与引擎启动

创建 /etc/docker/daemon.json，设置独立数据目录、vfs 存储驱动及 json-file 日志轮转。vfs 用于本次嵌套实验环境。dockerd --validate 返回 configuration OK 后，启动新引擎并核对 Client、Server、数据目录与日志驱动。

Docker 6085 补充实验 · 实际命令日志

04-first-configuration.txt · 原日志第 1—31 行 · UTC 时间按日志保留

```
Actual command: docker exec memory-game6085-v7-install-20261022 bash /lab/04-first-configuration.sh
Started UTC: 2026-10-22T06:33:28.496851+00:00

+ date -Iseconds
2026-10-22T06:33:28+00:00
+ mkdir -p /etc/docker
+ cat
+ cat /etc/docker/daemon.json
{
  "data-root": "/var/lib/docker-lab6085",
  "storage-driver": "vfs",
  "log-driver": "json-file",
  "log-opts": {"max-size": "10m", "max-file": "3"}
}
+ dockerd --validate --config-file=/etc/docker/daemon.json
configuration OK
+ nohup dockerd --config-file=/etc/docker/daemon.json
++ seq 1 60
+ for n in $(seq 1 60)
+ docker info
+ break
+ docker version --format 'Client={{.Client.Version}} Server={{.Server.Version}}'
Client=29.9.2 Server=29.9.2
+ docker info --format 'Name={{.Name}} Driver={{.Driver}} Root={{.DockerRootDir}} Cgroup=
{{.CgroupVersion}}'
Name=docker-lab6085 Driver=vfs Root=/var/lib/docker-lab6085 Cgroup=2
+ docker compose version
Docker Compose version v5.6.1
+ docker info --format '{{json .LoggingDriver}}'
"json-file"

Actual exit code: 0
```

图 A4 首次配置与引擎启动（操作日志）

A5 导入基础镜像并验证安装

为复用已下载的基础镜像，先在外层导出 Temurin、Registry 和 hello-world，再通过 docker load 导入本轮独立引擎。运行 hello-world 后出现 Hello from Docker，容器以退出码 0 结束，验证新安装的引擎能够实际运行容器。

Docker 6085 补充实验 · 实际命令日志

05-verify-installation.txt · 原日志第 1—31 行 · UTC 时间按日志保留

```
Actual command: docker exec memory-game6085-v7-install-20261022 bash /lab/05-verify-installation.sh
Started UTC: 2026-10-22T06:39:13.669479+00:00

+ date -Iseconds
2026-10-22T06:39:13+00:00
+ docker load -i /lab/base-images.tar
Loaded image: registry:3
Loaded image: hello-world:latest
Loaded image: eclipse-temurin:21-jre-jammy
+ docker run --name hello6085 hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

图 A5a 导入基础镜像并验证安装（操作日志，第 1 部分）

Docker 6085 补充实验 · 实际命令日志

05-verify-installation.txt · 原日志第 32—37 行 · UTC 时间按日志保留

```
+ docker ps -a --filter name=hello6085 --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}'
NAMES       IMAGE         STATUS
hello6085   hello-world   Exited (0) Less than a second ago

Actual exit code: 0
```

图 A5b 导入基础镜像并验证安装（操作日志，第 2 部分）

A6 创建网络、数据卷和镜像仓库

依次创建 memory-game6085-v7-net 网络、应用数据卷和仓库数据卷，再启动 Registry 容器。inspect 核对 bridge 网络及 local 数据卷驱动，仓库端口绑定补充环境内部的 127.0.0.1:5000，/v2/ 接口返回空 JSON 对象。

Docker 6085 补充实验 · 实际命令日志

06-create-resources.txt · 原日志第 1—26 行 · UTC 时间按日志保留

```
Actual command: docker exec memory-game6085-v7-install-20261022 bash /lab/06-create-resources.sh
Started UTC: 2026-10-22T06:39:21.776747+00:00

+ date -Iseconds
2026-10-22T06:39:21+00:00
+ docker network create memory-game6085-v7-net
fadef90bd60d90b2a879cc393ed6a7265856816f32eba9241acec06ba229b08c
+ docker volume create memory-game6085-v7-data
memory-game6085-v7-data
+ docker volume create memory-game6085-v7-registry
memory-game6085-v7-registry
+ docker run -d --name registry6085 --network memory-game6085-v7-net -p 127.0.0.1:5000:5000 -e
OTEL_TRACES_EXPORTER=none --mount type=volume,source=memory-game6085-v7-
registry,target=/var/lib/registry registry:3
99815ee6cf396d76f1ad930f5592dace565949340f3586658736afec2b7bf685
+ docker network inspect memory-game6085-v7-net --format 'Network={{.Name}} Driver={{.Driver}}
Subnet={{(index .IPAM.Config 0).Subnet}}'
Network=memory-game6085-v7-net Driver=bridge Subnet=172.19.0.0/16
+ docker volume inspect memory-game6085-v7-data memory-game6085-v7-registry --format 'Volume=
{{.Name}} Driver={{.Driver}}'
Volume=memory-game6085-v7-data Driver=local
Volume=memory-game6085-v7-registry Driver=local
+ docker ps --filter name=registry6085 --format 'table {{.Names}}\t{{.Image}}\t{{.Ports}}'
NAMES          IMAGE        PORTS
registry6085   registry:3   127.0.0.1:5000->5000/tcp
+ curl --retry 10 --retry-connrefused --retry-delay 1 -fsS http://127.0.0.1:5000/v2/
{}+ echo


Actual exit code: 0
```

图 A6 创建网络、数据卷和镜像仓库（操作日志）

A7 构建应用镜像并核对 JAR

采用源码包内的 Dockerfile.prebuilt 和配套 JAR，在新安装的引擎内构建 memory-game6085:1.0.2-v7。该步骤复用已打包的应用，不重新执行正文的 npm 与 Maven 构建。随后将容器内的 /app/app.jar 复制出来，分别计算它和配套 JAR 的 SHA-256，并逐字节比较两个文件，两项检查结果均一致。

```
docker build --pull=false --network=none -f Dockerfile.prebuilt -t memory-game6085:1.0.2-v7 .
```

Docker 6085 补充实验 · 实际命令日志

07-build-application.txt · 原日志第 1—31 行 · UTC 时间按日志保留

```
Actual command: docker exec memory-game6085-v7-install-20261022 bash /lab/07-build-application.sh
Started UTC: 2026-10-22T06:39:22.955601+00:00

+ date -Iseconds
2026-10-22T06:39:23+00:00
+ cd /lab
+ sha256sum dist-artifacts/memory-game6085.jar
2e97ff46f2acfc1bfe0411a22f3f61a3ef4b8958e4ba5fff609d0350853852a0  dist-artifacts/memory-
game6085.jar
+ docker build --pull=false --network=none -f Dockerfile.prebuilt -t memory-game6085:1.0.2-v7 .
+ tail -20 /lab/07-image-build-full.log

#4 [internal] load build context
#4 transferring context: 33.64MB 0.2s done
#4 DONE 0.3s

#6 [2/4] RUN groupadd --system garden && useradd --system --gid garden --home-dir /app garden
&& mkdir -p /app/data && chown -R garden:garden /app
#6 DONE 0.9s

#7 [3/4] WORKDIR /app
#7 DONE 0.6s

#8 [4/4] COPY --chown=garden:garden dist-artifacts/memory-game6085.jar /app/app.jar
#8 DONE 0.7s

#9 exporting to image
#9 exporting layers
#9 exporting layers 0.3s done
#9 writing image sha256:fdd03c0bf4b44385e2ddcc411693f5b541b9139f4971e20156aa5cb4f168ca70 done
#9 naming to docker.io/library/memory-game6085:1.0.2-v7 done
#9 DONE 0.3s
+ docker image inspect memory-game6085:1.0.2-v7 --format 'Image={{.Id}} User={{.Config.User}}'
```

图 A7a 构建应用镜像并核对 JAR（操作日志，第 1 部分）

Docker 6085 补充实验 · 实际命令日志

07-build-application.txt · 原日志第 32—43 行 · UTC 时间按日志保留

```
Image=sha256:fdd03c0bf4b44385e2ddcc411693f5b541b9139f4971e20156aa5cb4f168ca70 User=garden
+ docker create --name jarcheck6085 memory-game6085:1.0.2-v7
c6f060f8dcdfee48dbd4d735fa1a09e6388a72f70dba5eecf984fa0815266061
+ docker cp jarcheck6085:/app/app.jar /lab/container-app.jar
+ sha256sum /lab/container-app.jar /lab/dist-artifacts/memory-game6085.jar
2e97ff46f2acfc1bfe0411a22f3f61a3ef4b8958e4ba5fff609d0350853852a0  /lab/container-app.jar
2e97ff46f2acfc1bfe0411a22f3f61a3ef4b8958e4ba5fff609d0350853852a0  /lab/dist-artifacts/memory-
game6085.jar
+ cmp /lab/container-app.jar /lab/dist-artifacts/memory-game6085.jar
Container JAR matches the submitted JAR byte-for-byte
+ echo 'Container JAR matches the submitted JAR byte-for-byte'

Actual exit code: 0
```

图 A7b 构建应用镜像并核对 JAR（操作日志，第 2 部分）

A8 配置应用容器并保存成绩

启动应用容器时明确指定网络、18085:8080 端口映射、DATA_DIR 和数据卷挂载。健康接口返回 UP，运行用户为 garden，UID/GID 为 999。将本机 127.0.0.1:18089 的访问请求转发到隔离环境的 18085 端口后，通过浏览器打开该地址，完成一局轻松难度游戏并保存成绩。

本次保存的记录编号为 1，昵称为“6085 补充实验”，成绩为 10 步、4737 ms。首次健康检查时应用尚未就绪，连接失败；加入重试后，健康接口返回 UP。

Docker 6085 补充实验 · 实际命令日志

08-run-application.txt · 原日志第 1—27 行 · UTC 时间按日志保留

```
Actual command: docker exec memory-game6085-v7-install-20261022 bash /lab/08-run-application.sh
Started UTC: 2026-10-22T06:43:24.613282+00:00

+ date -Iseconds
2026-10-22T06:43:24+00:00
+ docker run -d --name garden6085 --network memory-game6085-v7-net -p 18085:8080 -e
DATA_DIR=/app/data --mount type=volume,source=memory-game6085-v7-data,target=/app/data memory-
game6085:1.0.2-v7
186285d65c93d996c49cc20dadb1ad1c5f54c47382af3a4f737009d14dd36904
+ curl --retry 30 --retry-all-errors --retry-delay 1 -fsS http://127.0.0.1:18085/api/health
curl: (56) Recv failure: Connection reset by peer
curl: (56) Recv failure: Connection reset by peer
curl: (56) Recv failure: Connection reset by peer
{"status":"UP","database":"SQLite","application":"memory-game6085"}+ echo
+ docker ps --filter name=garden6085 --format 'table {{.Names}}\t{{.Image}}\t{{.Ports}}'

NAMES        IMAGE                      PORTS
garden6085   memory-game6085:1.0.2-v7   0.0.0.0:18085->8080/tcp, [::]:18085->8080/tcp
+ docker inspect garden6085 --format 'Network={{json .NetworkSettings.Networks}}'
Network={"memory-game6085-v7-net":
{"IPAMConfig":null,"Links":null,"Aliases":null,"DriverOpts":null,"GwPriority":0,"NetworkID":"fadef9
0bd60d90b2a879cc393ed6a7265856816f32eba9241acec06ba229b08c","EndpointID":"5a8dc1c743b08d7b0e7c76fca
ba5ee1bace1b27599b11e25912764962595f3f8","Gateway":"172.19.0.1","IPAddress":"172.19.0.3","MacAddres
s":"b6:c8:5e:34:a5:a2","IPPrefixLen":16,"IPv6Gateway":"","GlobalIPv6Address":"","GlobalIPv6PrefixLe
n":0,"DNSNames":["garden6085","186285d65c93"]}}
+ docker inspect garden6085 --format '{{range .Mounts}}Volume={{.Name}} Destination=
{{.Destination}}{{end}}'
Volume=memory-game6085-v7-data Destination=/app/data
+ docker exec garden6085 id
uid=999(garden) gid=999(garden) groups=999(garden)
+ curl -fsS 'http://127.0.0.1:18085/api/leaderboard?difficulty=easy'
{"scores":[],"total":0,"bestDurationMs":null}+ echo


Actual exit code: 0
```

图 A8a 配置应用容器并保存成绩（操作日志）

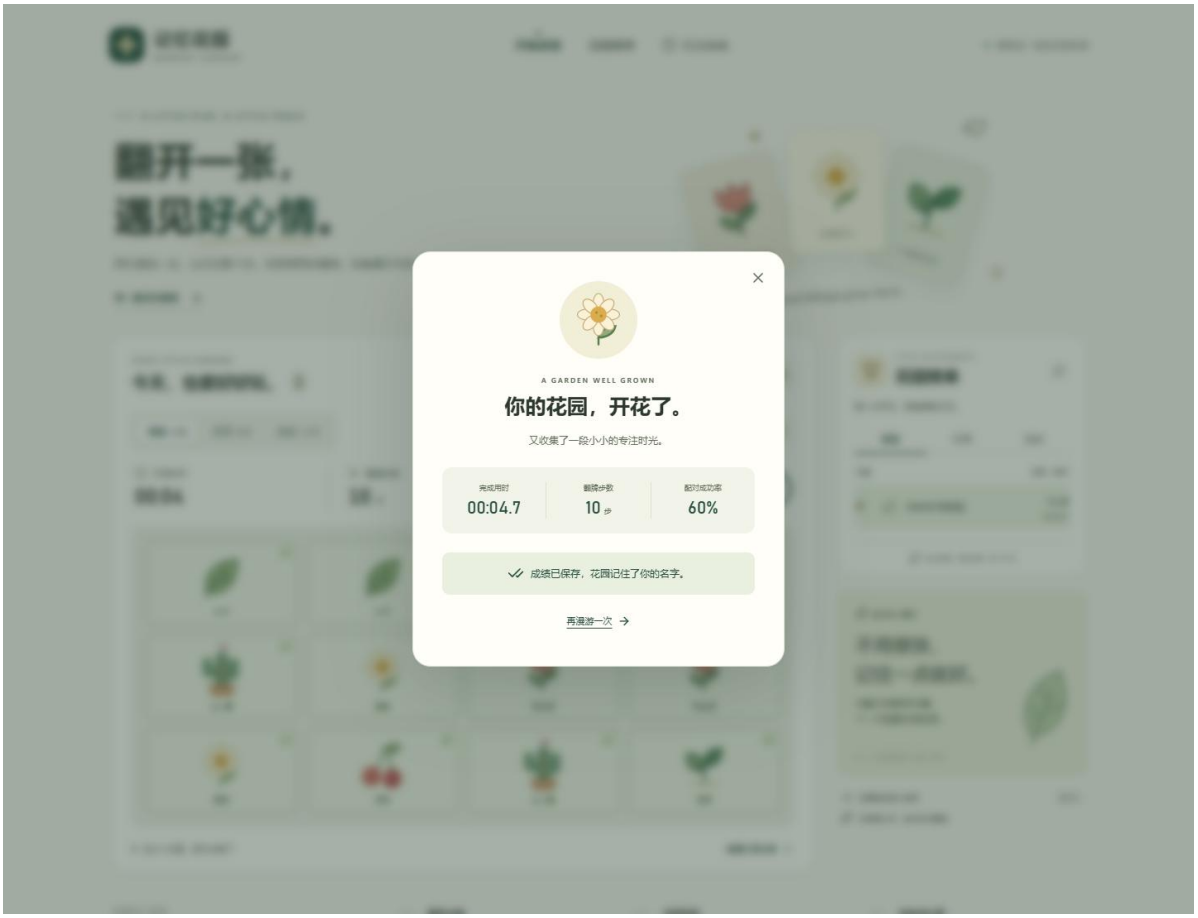


图 A8b 浏览器完成游戏并保存成绩

A9 标记并发布镜像

保存原容器标识和非空排行榜后，将镜像标记为 localhost:5000/memory-game6085:1.0.2-v7，并推送到本轮仓库。push 输出 Pushed 和镜像摘要，仓库标签接口返回 1.0.2-v7。

Docker 6085 补充实验 · 实际命令日志

09-publish-image.txt · 原日志第 1—31 行 · UTC 时间按日志保留

```
Actual command: docker exec memory-game6085-v7-install-20261022 bash /lab/09-publish-image.sh
Started UTC: 2026-10-22T06:46:37.388365+00:00

+ date -Iseconds
2026-10-22T06:46:37+00:00
+ docker inspect garden6085 --format '{{.Id}}'
+ tee /lab/before-container.txt
186285d65c93d996c49cc20dadb1ad1c5f54c47382af3a4f737009d14dd36904
+ curl -fsS 'http://127.0.0.1:18085/api/leaderboard?difficulty=easy'
+ tee /lab/before-leaderboard.json
{"scores":[{"id":1,"nickname":"6085补充实
验","difficulty":"easy","moves":10,"durationMs":4737,"createdAt":"2026-10-
22T06:45:40.193313517Z"}],"total":1,"bestDurationMs":4737}+ echo
+ grep -q '"nickname"' /lab/before-leaderboard.json

+ docker tag memory-game6085:1.0.2-v7 localhost:5000/memory-game6085:1.0.2-v7
+ docker push localhost:5000/memory-game6085:1.0.2-v7
+ tail -12 /lab/09-push-full.log
34af552be181: Waiting
129f77d88403: Waiting
98bd9880f810: Waiting
5f70bf18a086: Pushed
82b978d0aab5: Pushed
2302df99c4d0: Pushed
aa4b3397fa95: Pushed
2c7823f5b0d8: Pushed
34af552be181: Pushed
129f77d88403: Pushed
98bd9880f810: Pushed
1.0.2-v7: digest: sha256:bf7de8b8e400c33013aa3bacbb96e206df824dcf8346b0db0468023d390fd20e size:
1994
+ curl -fsS http://localhost:5000/v2/memory-game6085/tags/list
{"name":"memory-game6085","tags":["1.0.2-v7"]}
```

图 A9a 标记并发布镜像（操作日志，第 1 部分）

Docker 6085 补充实验 · 实际命令日志

09-publish-image.txt · 原日志第 32—34 行 · UTC 时间按日志保留

```
+ echo

Actual exit code: 0
```

图 A9b 标记并发布镜像（操作日志，第 2 部分）

A10 删除本地标签后重新拉取

移除本轮应用容器和 JAR 校验容器，再删除应用镜像的两个本地标签。通过 inspect 确认两个本地标签均不存在，并检查筛选后的镜像列表中已无 memory-game6085，再从 Registry 拉取镜像。拉取返回 Downloaded newer image，目标镜像摘要与 push 一致。拉取日志中部分镜像层显示 Already exists，说明这些层仍保留在本地；应用镜像的两个本地标签在拉取前均已删除。

Docker 6085 补充实验 · 实际命令日志

10-remove-and-pull.txt · 原日志第 1—31 行 · UTC 时间按日志保留

```
Actual command: docker exec memory-game6085-v7-install-20261022 bash /lab/10-remove-and-pull.sh
Started UTC: 2026-10-22T06:46:43.628882+00:00

+ date -Iseconds
2026-10-22T06:46:43+00:00
+ docker rm -f garden6085 jarcheck6085
garden6085
jarcheck6085
+ docker image rm memory-game6085:1.0.2-v7 localhost:5000/memory-game6085:1.0.2-v7
Untagged: memory-game6085:1.0.2-v7
Untagged: localhost:5000/memory-game6085:1.0.2-v7
Untagged: localhost:5000/memory-
game6085@sha256:bf7de8b8e400c33013aa3bacbb96e206df824dcf8346b0db0468023d390fd20e
Deleted: sha256:fdd03c0bf4b44385e2ddcc411693f5b541b9139f4971e20156aa5cb4f168ca70
+ docker image inspect memory-game6085:1.0.2-v7
+ docker image inspect localhost:5000/memory-game6085:1.0.2-v7
+ echo 'Both local application image tags are absent before pull'
+ docker image ls --filter 'reference=*memory-game6085*'
Both local application image tags are absent before pull
IMAGE   ID             DISK USAGE   CONTENT SIZE   EXTRA
+ docker pull localhost:5000/memory-game6085:1.0.2-v7
+ tail -15 /lab/10-pull-full.log
1.0.2-v7: Pulling from memory-game6085
dc541b8d0272: Already exists
f5c13179339d: Already exists
22d3391d8c49: Already exists
9ddd6de30055: Already exists
d18e088bcb32: Already exists
fa4f71bb68b5: Already exists
4f4fb700ef54: Already exists
a2c3041d80d8: Already exists
Digest: sha256:bf7de8b8e400c33013aa3bacbb96e206df824dcf8346b0db0468023d390fd20e
```

图 A10a 删除本地标签后重新拉取（操作日志，第 1 部分）

Docker 6085 补充实验 · 实际命令日志

10-remove-and-pull.txt · 原日志第 32—37 行 · UTC 时间按日志保留

```
Status: Downloaded newer image for localhost:5000/memory-game6085:1.0.2-v7
localhost:5000/memory-game6085:1.0.2-v7
+ docker image inspect localhost:5000/memory-game6085:1.0.2-v7 --format '{{json .RepoDigests}}'
["localhost:5000/memory-
game6085@sha256:bf7de8b8e400c33013aa3bacbb96e206df824dcf8346b0db0468023d390fd20e"]

Actual exit code: 0
```

图 A10b 删除本地标签后重新拉取（操作日志，第 2 部分）

A11 重建容器并核对持久化数据

使用拉取的仓库镜像创建 garden6085-pulled，重新挂载同一应用数据卷。新旧容器标识不同，完整排行榜 JSON 逐字节一致，输出 FullLeaderboardPreserved=True。最后在新容器的浏览器页面再次查看原成绩。

sha256:bf7de8b8e400c33013aa3bacbb96e206df824dcf8346b0db0468023d390fd20e

原容器标识前 12 位为 186285d65c93，新容器为 00a18a115fb4。重建前后保留同一条成绩，昵称、难度、步数、用时和创建时间均一致。

Docker 6085 补充实验 · 实际命令日志

11-recreate-and-verify.txt · 原日志第 1—31 行 · UTC 时间按日志保留

```
Actual command: docker exec memory-game6085-v7-install-20261022 bash /lab/11-recreate-and-verify.sh
Started UTC: 2026-10-22T06:46:46.129666+00:00

+ date -Iseconds
2026-10-22T06:46:46+00:00
+ docker run -d --name garden6085-pulled --network memory-game6085-v7-net -p 18085:8080 -e
DATA_DIR=/app/data --mount type=volume,source=memory-game6085-v7-data,target=/app/data
localhost:5000/memory-game6085:1.0.2-v7
00a18a115fb401d926753cae3bf408d85a755e90c69a55a7ce2a62718c604004
+ curl --retry 30 --retry-all-errors --retry-delay 1 -fsS http://127.0.0.1:18085/api/health
curl: (56) Recv failure: Connection reset by peer
curl: (56) Recv failure: Connection reset by peer
curl: (56) Recv failure: Connection reset by peer
{"status":"UP","database":"SQLite","application":"memory-game6085"}+ echo

+ docker inspect garden6085-pulled --format '{{.Id}}'
+ tee /lab/after-container.txt
00a18a115fb401d926753cae3bf408d85a755e90c69a55a7ce2a62718c604004
+ curl -fsS 'http://127.0.0.1:18085/api/leaderboard?difficulty=easy'
+ tee /lab/after-leaderboard.json
{"scores":[{"id":1,"nickname":"6085补充实
验","difficulty":"easy","moves":10,"durationMs":4737,"createdAt":"2026-10-
22T06:45:40.193313517Z"}],"total":1,"bestDurationMs":4737}+ echo
+ cmp /lab/before-leaderboard.json /lab/after-leaderboard.json

++ cat /lab/before-container.txt
++ cat /lab/after-container.txt
+ test 186285d65c93d996c49cc20dadb1ad1c5f54c47382af3a4f737009d14dd36904 '!='
00a18a115fb401d926753cae3bf408d85a755e90c69a55a7ce2a62718c604004
+ echo 'NewContainer=True; FullLeaderboardPreserved=True'
+ docker inspect garden6085-pulled --format '{{range .Mounts}}Volume={{.Name}} Destination=
{{.Destination}}{{end}}'
NewContainer=True; FullLeaderboardPreserved=True
Volume=memory-game6085-v7-data Destination=/app/data
+ docker exec garden6085-pulled id
uid=999(garden) gid=999(garden) groups=999(garden)
```

图 A11a 重建容器并核对持久化数据（操作日志，第 1 部分）

Docker 6085 补充实验 · 实际命令日志

11-recreate-and-verify.txt · 原日志第 32—32 行 · UTC 时间按日志保留

```
Actual exit code: 0
```

图 A11b 重建容器并核对持久化数据（操作日志，第 2 部分）



图 A11c 拉取镜像重建后的浏览器排行榜

本次在隔离 Ubuntu 环境中安装 Docker Engine，完成了应用镜像的构建、发布和拉取。使用拉取的镜像重建容器并挂载同一数据卷后，完整排行榜与重建前一致，浏览器中仍能查看原成绩。